

Jenkins Migration Process Documentation

0.1. Introduction

This document provides detailed instructions for migrating a Jenkins installation from Google Cloud Platform (GCP) to Amazon Web Services (AWS).

0.2. Prerequisites

0.2.1 Required Access

- SSH access to GCP instance
- AWS instance credentials.
- Administrative access to Jenkins

0.2.2 Required Tools

- SSH key pairs
- Sufficient storage space
- Terminal access

0.2.3 Environmental Requirements

- Stable network connection
- Backup storage capacity
- System permissions

<u>Current Version</u>	<u>New Version</u> <u>(Document Intended Upgrade)</u>
Jenkins 2.235.2	Jenkins 2.479.2
Centos-release-7-8.2003 (on GCP)	Centos-release-7-8.2003 (on GCP)
Ubuntu 24.04.1 LTS (on AWS)	Ubuntu 24.04.1 LTS (on AWS)

Jenkins Migration Guide: GCP PART

This part aims to assist user in migrating Jenkins from a Google Cloud Platform (GCP) instance running **CentOS** to another AWS server running **Ubuntu**. The process involves upgrading Jenkins and Java to their latest versions, creating a backup of the Jenkins files, and transferring this backup to the new server. By following these steps, you ensure to maintain the integrity of your Jenkins setup. The guide is structured to provide clear, step-by-step instructions for each phase of the migration.

1.1 Connect to GCP Instance via SSH

```
ssh -i <access key> <user>@<ip-address>
```

1.3 Verify Jenkins Status

```
jenkins --version
systemctl status jenkins
sudo systemctl stop jenkins
```

So here, we noticed that our currently installed Jenkins version and JAVA/JDK is way out of support. So we will now first upgrade our Jenkins to the latest LTS version and the compatible new java version using the given commands, (PS: these commands are for the centos version)

1.4 Update Repository

```
sudo wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/redhat-stable/jenkins.repo
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
sudo dnf upgrade
```

1.5 Upgrade Jenkins

```
# upgrading java
sudo dnf install fontconfig java-17-openjdk

# installing Jenkins
sudo dnf install Jenkins

# restarting our daemon
sudo systemctl daemon-reload
```

1.6 Verify and Restart Jenkins

```
jenkins --version
systemctl start jenkins
systemctl status jenkins
```

2. Backup Process

So, since we have upgraded our Jenkins and Java, we will now capture the files of Jenkins, i.e. create a backup file of the currently running Jenkins. So the trick we are playing is that we will backup the Jenkins, and capture all of the backend files of Jenkins, and then will shift the backed up zip file to the new server and merge with the same version of Jenkins, and hence we will have our Jenkins shifted from one server to another.

2.1 Stop Jenkins Service

```
sudo systemctl stop jenkins
```

2.2 Create and Download Backup

```
# Create backup
sudo tar -czf <location_and_name_of_backup_with_extension.tar.gz> /var/lib/jenkins

# Download from GCP Storage (if needed)
gcloud storage cp gs://<name_of_backup.tar.gz> .

# Verify backup
md5sum <name_of_backup.tar.gz>
```

2.3 Transfer Backup

```
# To local machine (optional)
scp -i <private_key> <username>@<ip-address>:<location_of_backup_file> '<new_file_location>'

# To AWS
scp -i <private_key> <location_of_backup_file> <username>@<ip-address>:/< new_file_location >
```

Alternatively, we can do all of the by creating a Bash script

Whatever steps we did above were done separately step wise, so to ease the process a bit, what we can do is create a bash script with the same steps and it will run them systematically and will do the task. To proceed,

Create a Bash script with the name:

```
jenkins_backup.sh
```

and run:

```
# Set variables
JENKINS_HOME="/var/lib/jenkins"
BACKUP_NAME="jenkins_backup_$(date +%Y%m%d)"
BACKUP_DIR=~/.jenkins_backup

# 1. Stop Jenkins service on GCP
echo "Stopping Jenkins service..."
sudo systemctl stop jenkins

# 2. Create backup directory if it doesn't exist
echo "Creating backup directory..."
mkdir -p ${BACKUP_DIR}

# 3. Backup essential Jenkins directories and files
echo "Creating backup of Jenkins configuration..."
sudo tar -czf ${BACKUP_DIR}/${BACKUP_NAME}.tar.gz \
  ${JENKINS_HOME}/jobs \
  ${JENKINS_HOME}/nodes \
  ${JENKINS_HOME}/plugins \
  ${JENKINS_HOME}/secrets \
  ${JENKINS_HOME}/users \
  ${JENKINS_HOME}/*.xml \
  ${JENKINS_HOME}/credentials.xml \
  ${JENKINS_HOME}/identity.key.enc \
  ${[ -d "${JENKINS_HOME}/.ssh" ] && echo "${JENKINS_HOME}/.ssh")

# 4. Fix backup file permissions
echo "Fixing backup file permissions..."
sudo chown $(whoami):$(whoami) ${BACKUP_DIR}/${BACKUP_NAME}.tar.gz

# 5. Verify backup was created successfully
if [ -f "${BACKUP_DIR}/${BACKUP_NAME}.tar.gz" ]; then
  echo "Backup completed successfully!"
  echo "Backup location: ${BACKUP_DIR}/${BACKUP_NAME}.tar.gz"
  echo "Backup size: $(du -h ${BACKUP_DIR}/${BACKUP_NAME}.tar.gz | cut -f1)"
else
  echo "Error: Backup creation failed!"
  exit 1
fi

# 6. Ask to restart Jenkins
echo "Do you want to restart Jenkins? (y/n)"
read response
if [ "$response" = "y" ]; then
  sudo systemctl start jenkins
  echo "Jenkins has been restarted"
else
  echo "Jenkins remains stopped"
fi
```

Run the following Command:

```
./jenkins_backup.sh
```

Eureka, a Backup copy has been made!

Jenkins Migration Guide: AWS PART

This part is designed to help user migrate Jenkins from a Google Cloud Platform (GCP) instance to an Amazon Web Services (AWS) EC2 instance running **Ubuntu**. The process involves setting up the AWS environment, installing the necessary dependencies, and transferring the Jenkins backup from the GCP instance. By following these steps, you can preserve the integrity of your Jenkins setup. The part provides detailed step-by-step instructions for each phase of the migration.

1.1 Login to AWS via SSH from a new terminal

```
ssh -i <access key> <user>@<ip-address>
```

1.2 Push Jenkins Backup from local machine

2. AWS Setup

Now we will prepare our AWS EC2 server for the Jenkins migration.

2.1 Update System

```
sudo apt update
```

2.2 Install Java

Supported version of the respective Jenkins

```
sudo apt install openjdk-17-jre
```

```
#Verify Jenkins version  
java -version
```

2.3 Install Jenkins

The same version we backed up on GCP server, in this case it is Jenkins 2.479.2

```
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \  
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key  
  
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]" \  
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \  
/etc/apt/sources.list.d/jenkins.list > /dev/null  
  
# Update and install Jenkins  
sudo apt-get update  
sudo apt-get install jenkins  
  
# Enable Jenkins service  
sudo systemctl enable jenkins
```

2.4 Configure Jenkins Directory

This is where we will unload the backed-up data

```
# Create Jenkins directory if it doesn't exist
sudo mkdir -p /var/lib/jenkins

# Set proper ownership
sudo chown jenkins:jenkins /var/lib/jenkins
```

3. Data Migration

3.1 Stop Jenkins Service

```
sudo systemctl stop jenkins
```

3.2 Extract Backup

To the configured Jenkins location

```
# Extract the backup to Jenkins directory
sudo tar -xzf < name_of_backup > -C /var/lib/jenkins

# Fix permissions
sudo chown -R jenkins:jenkins /var/lib/jenkins
sudo chmod -R 755 /var/lib/jenkins
```

4. Port Configuration (optional)

4.1 Modify Jenkins Port

We are doing so because we want our new Jenkins to be accessible via a Domain with SSL.

```
# Edit Jenkins configuration
sudo nano /etc/default/jenkins

# Add/modify the following line:
# HTTP_PORT=8181
```

5. Nginx Setup (optional)

Note: we got a SSL certificate issued by Let's Encrypt ACME challenge

5.1 Install Nginx

```
sudo apt install nginx
```

5.2 Configure Nginx

```
sudo nano /etc/nginx/sites-available/<domain_name>
```

Add the following configuration:

```
server {
    listen 80;
    listen 443 ssl;
    server_name <domain_name>;

    ssl_certificate /etc/letsencrypt/live/<domain_name>/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/<domain_name>/privkey.pem;

    location / {
        proxy_pass http://localhost:8181;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

5.3 Enable Site Configuration

```
sudo ln -s /etc/nginx/sites-available/<domain_name> /etc/nginx/sites-enabled/
```

6. SSL Configuration (SSL issued via let 'encrypt ACME challenge)

6.1 Create SSL Directory

```
sudo mkdir -p /etc/letsencrypt/live/<domain_name>
```

6.2 Configure SSL

```
# Move SSL certificates
sudo mv /home/ubuntu/fullchain.pem /etc/letsencrypt/live/<domain_name>/
sudo mv /home/ubuntu/privkey.pem /etc/letsencrypt/live/<domain_name>/

# Set proper permissions
sudo chown root:root /etc/letsencrypt/live/<domain_name>/fullchain.pem /etc/letsencrypt/live/<domain_name>/privkey.pem
sudo chmod 644 /etc/letsencrypt/live/<domain_name>/fullchain.pem /etc/letsencrypt/live/<domain_name>/privkey.pem
```

7. Final Steps

7.1 Test and Start Services

```
# Test Nginx configuration
sudo nginx -t

# Start services
sudo systemctl start jenkins
sudo systemctl restart nginx

# Verify services
sudo systemctl status jenkins
sudo systemctl status nginx
```

Verification Checklist

Before Migration:

- Jenkins version documented.
 - All required plugins listed.
 - Configuration files backed up.
 - Jobs and workspace data included in backup.
 - SSL certificates copied (if used)
-

After Migration:

- Backup file size matches source
 - All jobs are present.
 - Plugins restored.
 - System configurations matched.
 - SSL certificates properly installed.
-

Let's Connect: [Sannidhya Srivastava](#)

Drafted with ❤️ by [Sannidhya Srivastava](#)